Automated Software Testing (AST)

Complete Midterm Study Guide (Week 1 - 7) | Instructor: Mam Hira Asim

WEEK 1: INTRO TO AUTOMATED TESTING & KATALON

Manual vs. Automation Testing: Manual testing tester khud perform karta hai (slower, human error chance). Automation testing mein scripts aur tools (Selenium, Katalon) use hote hain repetitive tasks ko fast, reliable aur reusable banane ke liye.

> **Automation Testing Workflow (8 Steps):** 1. Identify Test Scenario → 2. Create Test Script → 3. Open & Configure App → 4. Locate UI Elements → 5. Perform Actions → 6. Verify Results → 7. Generate Test Results → 8. Maintain & Execute Tests.

Katalon Recorder: Ek lightweight browser extension for record-and-playback. *Limitations:* Complex logic (loops/conditions) handle nahi karta, parallel execution support nahi karta, aur reporting minimal hoti hai.

WEEK 2: SELENIUM WEBDRIVER & MAVEN

Maven: Java projects mein `pom.xml` ke zariye dependencies (`selenium-java`, `testng`) manage karta hai.
WebDriver: Browser automate karne ke liye interface hai.

Selenium Command	Action / Description
`driver.get("URL");`	Specific URL/Website open karna.
`driver.findElement(By...);`	Web page par ek element dhoondna.
`element.click();`	Button ya link par click karna.
`element.sendKeys("text");`	Text field/input box mein text enter karna.
`driver.getTitle();` / `getCurrentUrl();`	Page ka title ya mojooda URL get karna.
`driver.manage().window().maximize();`	Browser window ko maximize karna.
`driver.manage().window().minimize();`	Browser window ko minimize karna.
`driver.navigate().to("URL");`	Browser navigation (`.back()`, `.forward()` bhi hote hain).
`driver.quit();`	Tamam windows close aur session end karna (hamesha end par use karein).

Highlight Element Method (JavascriptExecutor): Element ko red border se highlight karne ke liye (Debugging ke liye mufeed):

```
public void highlightElement(WebElement element) throws InterruptedException {
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].style.border='6px solid red'", element);
Thread.sleep(3000); }
```

WEEK 3: DOM & WEB ELEMENT LOCATORS

DOM (Document Object Model): HTML code browser mein DOM tree banata hai. Selenium HTML nahi, balkay DOM ke elements (tags, attributes) ke sath interact karta hai.

Locator Type	Syntax & Description
ID (Fastest)	`By.id("email")` - Sab se reliable aur unique hota hai.
Name	`By.name("emailAddress")` - Form fields (text, radio) ke liye mufeed.
ClassName	`By.className("form-control")` - Class attribute se dhoondta hai.
Tag Name	`By.tagName("input")` - HTML tag se dhoondta hai (aksar `findElements` ke sath).
Link Text	`By.linkText("Contact us")` - `<a>` tag ke exact visible text se.
Partial Link Text	`By.partialLinkText("art")` - Hyperlink ke partial text se match karta hai.

CSS Selectors (Fast & Flexible)

> • **ID:** `#email`
> • **Class:** `.form-control`
> • **Attribute:** `input[name='userEmail']`
> • **Contains (*=):** `input[placeholder*='Email']` (Email word kahin bhi ho)
> • **Starts With (^=):** `input[placeholder^='Sub']` (Start mein Sub ho)
> • **Ends With ($=):** `textarea[placeholder$='Message Here']` (End mein ho)

XPath Locators (Robust but Slower)

> **Basic Syntax:** `//tagname[@attribute='value']`
> • **By Attribute:** `//input[@id='search_product']`
> • **By Exact Text:** `//button[text()='Login']` ya `//h2[text()='Brands']`
> • **By Contains:** `//*[contains(text(), 'View Product')]` (Partial text match)
> • **By Starts-With:** `//input[starts-with(@id, 'search')]`

WEEK 4: WEB ELEMENT INTERACTION

Automation ka standard pattern: **Locate** → **Interact** → **Verify**.

Verification Method	Purpose (Returns Value)
`getText()`	Element ka visible string text nikalta hai.
`getAttribute("val")`	Element ke kisi specific attribute (jaise placeholder) ki value deta hai.
`isDisplayed()`	Returns true agar element screen par visible ho.
`isSelected()`	Returns true agar Checkbox/Radio button ticked/selected ho.
`isEnabled()`	Returns true agar button ya field active (clickable/typable) ho.

WEEK 5: ACTIONS CLASS & WINDOW HANDLES

1. Actions Class (Mouse & Keyboard)

Complex interactions ke liye `Actions` class use hoti hai. Action ko execute karne ke liye end par `.perform()` lagana lazmi hai. Chain banani ho toh `.build().perform()` use karte hain.

```
Actions actions = new Actions(driver); // Hover and Click
actions.moveToElement(element).click().perform(); // Double Click & Right Click (Context
Click) actions.doubleClick(element).perform(); actions.contextClick(element).perform(); //
Scrolling using Keys and amount actions.sendKeys(Keys.PAGE_DOWN).perform();
actions.scrollByAmount(0, 200).perform(); // scroll down 200px
```

2. Multiple Tabs and Windows

Selenium har tab/window ko ek unique ID (Window Handle) assign karta hai. Naye tab mein move karne ke liye pehle Handle get karna parta hai.

```
String mainWindow = driver.getWindowHandle(); // Current Tab ID // Open new tab
(Selenium 4 feature) driver.switchTo().newWindow(WindowType.TAB); // ya WindowType.WINDOW for
new window Set<String> allWindows = driver.getWindowHandles(); // All open tabs IDs for
(String window : allWindows) { if (!window.equals(mainWindow)) {
driver.switchTo().window(window); // Switch focus to new tab } }
```

3. Synchronization (Waits)

`Thread.sleep()` hardcoded pause hai jo slow test execution ka bais banta hai. Selenium built-in waits provide karta hai:

- **Implicit Wait (Global):** Setup mein ek dafa define hota hai. Element find karne ke liye maximum wait time set karta hai.
  `driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));`

- **Explicit Wait (Specific Element):** Sirf kisi khaas condition (visible/clickable) ke meet hone ka wait karta hai.
  `WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));`
  `wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("btn")));`

WEEK 6: TESTNG ASSERTIONS, CONFIG & REPORTING

TestNG Annotation	Execution Timing & Purpose
`@BeforeClass`	Class start hone par sirf ek dafa (Jaise WebDriver setup).
`@BeforeMethod`	Har `@Test` se pehle chalta hai (Jaise URL open karna).
`@Test`	Actual test case ko represent karta hai.
`@AfterMethod`	Har `@Test` method ke baad chalta hai.
`@AfterClass`	Saray test methods khatam hone par aakhir mein (Jaise driver.quit).

Assertion (Validation)	Meaning (If fails, test stops immediately)
`Assert.assertTrue(condition)`	Condition LAZMI True honi chahiye.
`Assert.assertEquals(act, exp)`	Actual text expected text ke bilkul barabar hona chahiye.
`Assert.assertNotNull(element)`	Element screen par exist karta ho (Null na ho).
`assertTrue(str.contains("..."))`	String ke andar specific word mojood ho (Partial match).
`assertTrue(url.matches(".*"))`	String kisi specific regex pattern se match karti ho.

Study Resources Provided by Mam:
TestNG XML Suite bananay aur Test-Output Reports generate karne ke practical steps ke liye in official lecture links ko dekhein:

► Create TestNG XML File ► Generate test-output Reports

WEEK 7: POM, PAGEFACTORY & CONFIGREADER

Page Object Model (POM): UI Elements (Locators) aur Test Logic ko alag classes mein rakhne ka design pattern hai. UI tabdeel hone par sirf Page Class update karni parti hai, tests nahi.

PageFactory Annotations: `findElement` ki jagah use hote hain. Page constructor mein `PageFactory.initElements(driver, this);` lazmi hai.
• `@FindBy(id="userName")` : Ek single element ko locate karne ke liye.
• `@FindBys({@FindBy(class="a"), @FindBy(tag="b")})` : Element tab match hoga jab **DONO (AND)** conditions true hon gi.
• `@FindAll({@FindBy(id="a"), @FindBy(name="b")})` : Element tab match hoga jab **KOI EK (OR)** condition true ho gi.

```
     // ConfigReader Usage (Centralized Configuration) // data.properties mein:
baseUrl=http://example.com String baseUrl = ConfigReader.getProperty("baseUrl");
driver.get(baseUrl);
```